



TEST PLAN REPORT

Chef Bono

CMPE 492 /April 2026

Team	Chef Bono Team
Members	Berkan Gökgöz, Buse Şahin, Yelda Sıla Mumcu, Eren Serdar
Course	CMPE 492
Date	April 2026
Version	1.0

1. Introduction

1.1 Purpose

This Test Plan defines the scope, approach, resources, and schedule for the testing activities of Chef Bono — an AI-powered, voice-controlled cooking assistant developed as part of CMPE 492. The goal is to verify that each subsystem meets its specified requirements and that the integrated system delivers a reliable, responsive, and user-friendly experience.

1.2 Scope

Testing covers all subsystems documented in the Low-Level Design (LLD v2.0), including:

- AI Logic (ai_utils.py): Chat, AIUtils, function calling, embeddings, SQL generation
- Speech-to-Text (stt.py): audio capture, Whisper transcription, fallback handling
- Text-to-Speech (tts.py): Piper TTS synthesis and local audio playback
- Database & Recipe Management: Database, Recipe, row_adder ingestion pipeline
- User Interface (Interface.py): BasicInterface event handling and restart flow
- Main Orchestrator (__main__.py): end-to-end session loop and intent routing

1.3 Definitions and Acronyms

Term	Definition
STT	Speech-to-Text — converting microphone audio to a text string
TTS	Text-to-Speech — synthesising audio from a text string
LLD	Low-Level Design document (v2.0, Chef Bono)
UAT	User Acceptance Testing
UT	Unit Test
IT	Integration Test
ST	System Test
PT	Performance Test

2. Features to be Tested

The following features are in scope for this test plan:

- Multi-turn conversational state management (Chat class)
- Gemini API integration: prompting, function calling, schema-constrained JSON output
- Token counting and context window management
- PCA-reduced embedding generation and vector similarity search
- SQL query generation from natural language input
- Turkish speech recognition using Whisper base model
- Turkish text-to-speech synthesis using Piper TTS (local, offline)
- SQLite CRUD operations for recipe metadata
- Vector search on recipes_vec virtual table
- Batch ingestion pipeline (row_adder): flag enrichment, validation, DB insertion
- Recipe domain object construction via factory method
- AI-generated cooking step generation
- Intent routing in main orchestrator (generate_sql, select_recipe, follow_preparation, respond_to_user)
- BasicInterface UI event handling (continue, stop, restart, exit)

2.1 Features NOT Tested in This Plan

- Camera integration (camera.py) — deferred to Phase 2; image path is currently hardcoded
- select_recipe_manually() and display_image_category() UI stubs — not yet implemented
- Cloud deployment or multi-user concurrency — out of project scope

3. Testing Methodology

3.1 Unit Testing

Unit tests verify individual methods and functions in isolation, using mocks for external dependencies (Gemini API, SQLite, file system). Each test is identified by a UT-XXX label. The Python unit test framework is used throughout the system.

3.2 Integration Testing

Integration tests verify the interactions between two or more subsystems — for example, the pipeline from STT transcription through intent detection to TTS playback. Integration tests use the real database and API where feasible, with recorded API responses used for offline runs.

3.3 System Testing

System tests execute complete end-to-end scenarios from a user's perspective, driving the application through the main conversation loop. These tests confirm that all subsystems collaborate correctly to deliver the target user experience.

3.4 Performance Testing

Performance tests measure latency and throughput for the most time-sensitive operations: STT transcription, TTS synthesis, Gemini API response, vector search, and batch ingestion. Targets are defined per test case.

3.5 User Acceptance Testing (UAT)

UAT is conducted with real users (team members acting as first-time users, plus external volunteers where available). Testers follow realistic usage scenarios without scripted prompts. Satisfaction is measured via a short survey.

4. Test Environment

Component	Specification
Operating System	macOS (primary development platform)
Python Version	3.10+
Key Libraries	openai-whisper, piper-tts, google-genai, sqlite3, sqlite-vec, sklearn, sounddevice, scipy, numpy
Database	SQLite 3 — recipes.db (~22,000 recipes)
AI Model	Google Gemini 2.5 Flash Lite (via API)
STT Model	OpenAI Whisper base checkpoint (local)
TTS Engine	Piper TTS — en_US-hfc_female-medium (local, offline)
Network	Internet required for Gemini API calls; STT and TTS are fully offline
Test Runner	Python unittest (unit/integration); manual for UAT

5. Unit Test Cases

Unit tests target individual classes and functions. Mocks are used for Gemini API and file I/O where noted.

Test ID	Test Name	Description / Steps	Expected Result	Actual Result	Status
UT-001	Chat History Append	Call <code>_update_chat_history()</code> with a string prompt and string output; verify <code>chat_history</code> has 2 new entries.	<code>chat_history</code> gains ('user', [Part]) and ('assistant', [Part]) pairs.	Same as expected	Completed
UT-002	Chat History Pop	Append 2 turns, call <code>_pop_convo()</code> ; verify last 2 entries are removed.	<code>chat_history</code> length decreases by 2.	Same as expected	Completed
UT-003	Chat Pop Empty Guard	Call <code>_pop_convo()</code> on an empty Chat; verify it does not raise and prints a message.	No exception; warning printed.	Same as expected	Completed
UT-004	Is List of Parts – True	Pass a list of valid Part objects to <code>_is_list_of_parts()</code> ; check return value.	Returns True.	Same as expected	Completed
UT-005	Is List of Parts – False	Pass a list containing a plain string to <code>_is_list_of_parts()</code> .	Returns False.	Same as expected	Completed
UT-006	Token Count Text	Call <code>_get_token_count()</code> with a single text Part; verify non-zero integer returned.	Returns integer > 0.	Same as expected	Completed
UT-007	Image Token Estimate Fallback	Pass corrupted bytes to <code>_estimate_image_token_cost()</code> ; confirm fallback value.	Returns 258.	Same as expected	Completed
UT-008	Context Limit Check	Construct a prompt whose token count exceeds <code>input_token_limit</code> ; call <code>_check_for_exceptions()</code> .	Returns True and prints overflow message.	Same as expected	Completed
UT-009	<code>to_gemini_text</code> – dict	Pass a Python dict to <code>to_gemini_text()</code> ; verify result is a Part with JSON string.	Part.text contains valid JSON string.	Same as expected	Completed
UT-010	<code>to_gemini_text</code> – None	Pass None to <code>to_gemini_text()</code> .	Returns Part with empty string.	Same as expected	Completed
UT-011	<code>to_gemini_image</code> – valid path	Pass a valid JPEG path to <code>to_gemini_image()</code> .	Returns a Part with image bytes and correct MIME type.	Same as expected	Completed
UT-012	<code>to_gemini_image</code> – invalid path	Pass a non-existent file path to <code>to_gemini_image()</code> .	Returns None; no exception propagated.	Same as expected	Completed

Test ID	Test Name	Description / Steps	Expected Result	Actual Result	Status
UT-013	serialize_f32 – valid vector	Pass a list of 256 floats to <code>serialize_f32()</code> ; verify output length.	Returns bytes of length $256 * 4 = 1024$.	Same as expected	Completed
UT-014	serialize_f32 – non-float input	Pass a list containing a string to <code>serialize_f32()</code> .	Raises <code>struct.error</code> .	Same as expected	Completed
UT-015	STT <code>record_audio</code> file creation	Call <code>record_audio()</code> with <code>duration=1s</code> ; verify output WAV file exists on disk.	<code>input.wav</code> created at the specified path.	Same as expected	Completed
UT-016	STT <code>speech_to_text</code> returns string	Provide a known WAV file with clear Turkish speech; call <code>speech_to_text()</code> .	Returns non-empty Turkish string.	Same as expected	Completed
UT-017	STT fallback on empty transcription	Provide a silent WAV file; call <code>speech_to_text()</code> .	Returns fallback string 'Seni anlayamadim'.	Same as expected	Completed
UT-018	TTS <code>output.mp3</code> creation	Call <code>text_to_speech()</code> with a short Turkish string; verify <code>output.mp3</code> is created.	<code>output.mp3</code> exists and <code>size > 0</code> .	Same as expected	Completed
UT-019	Database <code>execute_query</code> – valid SQL	Call <code>execute_query()</code> with <code>SELECT * FROM recipes LIMIT 1</code> ; check list returned.	Returns list with at least one row dict.	Same as expected	Completed
UT-020	Database <code>execute_query</code> – bad SQL	Call <code>execute_query()</code> with malformed SQL.	Returns <code>None</code> ; error printed; no exception propagated.	Same as expected	Completed
UT-021	Database <code>get_recipe</code> – known name	Call <code>get_recipe()</code> with a recipe name known to exist in DB.	Returns non-empty dict with 'name' key matching query.	Same as expected	Completed
UT-022	Database <code>get_recipe</code> – unknown name	Call <code>get_recipe()</code> with a name not in the DB.	Returns empty dict.	Same as expected	Completed
UT-023	Recipe <code>build()</code> field mapping	Call <code>build()</code> with a fully-populated <code>query_result</code> dict; inspect all Recipe attributes.	All 25 required fields mapped correctly to Recipe object.	Same as expected	Completed
UT-024	<code>get_steps</code> returns list	Call <code>get_steps()</code> on a valid Recipe object; verify return type.	Returns non-empty list of strings.	Same as expected	Completed
UT-025	<code>fix_exceptions</code> – 429 retry	Mock a 429 response from the Gemini API; verify <code>_fix_exceptions()</code> sleeps and returns <code>True</code> .	Sleeps for <code>retryDelay</code> seconds; returns <code>True</code> .	Same as expected	Completed

6. Integration Test Cases

Integration tests verify cross-subsystem interactions using real components where possible.

Test ID	Test Name	Description / Steps	Expected Result	Actual Result	Status
IT-001	STT → AIUtils intent detection	Record a Turkish utterance asking for a recipe; pipe transcription into <code>get_intended_function_and_argument()</code> .	Returns ('generate_sql', {query_params}) tuple.	Same as expected	Completed
IT-002	AIUtils → Database SQL execution	Call <code>generate_sql_query()</code> with a user query; pass resulting SQL to <code>database.execute_query()</code> .	Non-empty list of recipe rows returned.	Same as expected	Completed
IT-003	Database → Recipe build pipeline	Call <code>get_recipe()</code> then pass result to <code>build()</code> ; verify <code>Recipe.steps</code> is populated.	Recipe object with all fields and non-empty steps list.	Same as expected	Completed
IT-004	AIUtils → TTS response playback	Call <code>talk_to_ai()</code> with a greeting; pass returned <code>chef_response</code> to <code>tts.play()</code> .	<code>output.mp3</code> created and played without error.	Since we use threading, some problems occurs when recalling tts	Complete / in-progress
IT-005	select_recipe intent → Recipe construction	Simulate user selecting recipe index 0 from <code>latest_query_result</code> ; verify <code>Recipe</code> object stored in main module.	recipe variable holds a fully built <code>Recipe</code> ; TTS confirmation plays.	Same as expected	Completed
IT-006	follow_preparation intent → step narration	With a <code>Recipe</code> selected, simulate user asking for next step; call <code>follow_preparation()</code> .	Returns <code>cooking_instruction</code> and <code>keep_convo_alive=True</code> ; TTS plays instruction.	Same as expected	Completed
IT-007	row_adder ingestion pipeline	Run the <code>__main__</code> block of <code>row_adder.py</code> on a batch of 5 test recipes; verify DB insertion.	5 rows appear in recipes table; 5 rows in <code>recipes_vec</code> with non-zero embeddings.	Same as expected	Completed
IT-008	Vector similarity search round-trip	Embed a query text; run generated SQL with <code>MATCH</code> clause on <code>recipes_vec</code> ; verify ranked results.	Returns recipes ordered by embedding similarity; top result semantically matches query.	It works but sometimes an unexpected matrix error occurs	Complete/in-progress

Test ID	Test Name	Description / Steps	Expected Result	Actual Result	Status
IT-009	Chat history persistence across turns	Execute two sequential talk_to_ai() calls on the same Chat instance; inspect chat_history.	chat_history contains 4 entries (2 user + 2 assistant turns).	Same as expected	Completed
IT-010	BasicInterface → on_restart reinitialisation	Click Restart button; verify root is destroyed and a new window opens.	New tkinter window rendered; no TclError raised.	Same as expected	Completed

7. System Test Cases

System tests drive complete user scenarios through the application from speech input to audio output.

Test ID	Test Name	Description / Steps	Expected Result	Actual Result	Status
ST-001	End-to-end recipe search session	User says 'I want a vegetarian pasta recipe'. System records, transcribes, detects intent, queries DB, narrates results via TTS.	User hears list of matching recipes within 10 seconds.	Same as expected	Completed
ST-002	End-to-end recipe selection and guidance	Following ST-001, user selects recipe 1. System builds Recipe and begins step-by-step guidance.	User hears first cooking step; subsequent steps narrated on request.	Same as expected	Completed
ST-003	General conversation fallback	User says 'Hello, who are you?' without requesting a recipe.	System routes to <code>respond_to_user</code> ; TTS plays a chef-persona greeting.	Same as expected	Completed
ST-004	Kitchen image analysis	Place ingredients on camera view; trigger <code>analyze_kitchen_image()</code> ; verify structured JSON output.	Returns <code>food_items</code> , <code>supplies</code> , and <code>tips</code> fields; TTS narrates tips.	Same as expected	Completed
ST-005	Context window overflow handling	Carry on a conversation until <code>chat_history</code> exceeds 80% of <code>input_token_limit</code> ; observe shrink behaviour.	<code>_shrink_chat_history()</code> pops old turns; conversation continues without API error.	Same as expected	Completed
ST-006	Session restart from UI	User clicks Restart button mid-session; start a new recipe search.	All session variables reset; fresh chat started; system fully functional.	Same as expected	Completed
ST-007	Multi-dietary filter query	User requests 'a vegan, gluten-free, high-protein breakfast recipe'.	SQL query includes correct WHERE clause flags; at least one recipe returned if available.	Same as expected	Completed
ST-008	No matching recipes graceful response	Query for an extremely rare combination expected to return 0 results.	System informs user no recipes were found; does not crash.	Same as expected	Completed

8. Performance Test Cases

Test ID	Test Name	Description / Steps	Expected Result	Actual Result	Status
PT-001	STT transcription latency	Record a 5-second audio clip; measure time from end of recording to return of transcribed string.	Transcription completes within 3 seconds on target hardware.	Same as expected	Completed
PT-002	TTS synthesis latency	Call tts.play() with a 50-word string; measure time to first audio output.	Audio begins playing within 2 seconds.	Work in progress	Work in progress
PT-003	Gemini API response time	Call talk_to_ai() 10 times sequentially; log and average response times.	Average response time < 5 seconds; no request exceeds 15 seconds.	Same as expected	Completed
PT-004	Vector search query performance	Execute a MATCH query on recipes_vec with 22,000 records; measure execution time.	Query returns results within 5 second.	Same as expected	Completed

9. User Acceptance Test Cases

Test ID	Test Name	Description / Steps	Expected Result	Actual Result	Status
UAT-001	New user onboarding	A user with no prior knowledge of Chef Bono completes a full recipe search and cooking session without assistance.	User successfully cooks a dish guided solely by voice prompts.	Same as expected	Completed
UAT-002	Turkish language comprehension	Native Turkish speaker uses natural language queries (not scripted); system correctly identifies intent.	Intent detected correctly for at least 80% of queries.	Work in progress	Work in progress
UAT-003	Dietary filter trust	A vegetarian user requests vegetarian recipes and inspects 5 returned results.	All 5 results are correctly flagged as vegetarian.	Same as expected	Completed

10. Roles and Responsibilities

Team Member	Role	Assigned Test IDs
Berkan Gökgöz	AI Logic & Integration Lead	UT-001–012, IT-001–005, ST-003–005, PT-003
Eren Serdar	Database & Ingestion Lead	UT-013–014, UT-019–022, IT-007–009, PT-004–005, ST-007–008
Yelda Sıla Mumcu	STT/TTS & System Testing Lead	UT-015–018, IT-004, IT-006, ST-001–002, ST-006, PT-001–002
Buse Şahin	UI, UAT & Recipe Pipeline Lead	UT-023–025, IT-010, ST-004, UAT-001–004

All team members are responsible for:

- Reporting defects in the shared issue tracker within 24 hours of discovery
- Retesting fixed defects before marking them as closed
- Attending the weekly test review meeting

11. Test Schedule

Phase	Dates	Responsible	Activities
Week 1	Apr 14–18	Berkan, Serdar	Set up test environment; write unit test harness; execute UT-001–025
Week 2	Apr 21–25	All	Execute integration tests IT-001–010; fix blocking defects
Week 3	Apr 28 – May 2	Yelda, Buse	Execute system tests ST-001–008; performance tests PT-001–005
Week 4	May 5–9	Buse, Yelda	User Acceptance Testing UAT-001–004 with real users
Week 5	May 12–16	All	Defect triage, regression testing, final report compilation

12. Control Procedures

12.1 Defect Management

- All defects are logged in the team's shared issue tracker with a severity level (Critical, Major, Minor, Trivial).
- Critical defects (system crash, data corruption) must be fixed before moving to the next test phase.
- Major defects (incorrect output, feature broken) are addressed within 48 hours.
- Minor and trivial defects are addressed before the final submission.

12.2 Entry and Exit Criteria

Unit & Integration Tests

- Entry: Test environment set up; code merged to main branch; test data prepared.
- Exit: All test cases executed; no open Critical or Major defects; pass rate $\geq 90\%$.

System & Performance Tests

- Entry: All integration tests passed.
- Exit: All ST and PT cases executed; latency targets met; no open Critical defects.

UAT

- Entry: System tests passed; real users recruited.
- Exit: All UAT cases completed; average satisfaction $\geq 3.5/5$; all Critical defects resolved.

12.3 Test Suspension and Resumption

Testing may be suspended if more than 3 Critical defects are open simultaneously, or if a subsystem is unavailable (e.g., Gemini API outage). Testing resumes once the blocking issues are resolved and a re-entry review is completed by the team.

13. Risks and Mitigations

Risk ID	Risk Description	Likelihood	Impact	Mitigation
R-001	Gemini API rate limiting (429 errors)	High	Medium	Implement exponential backoff in <code>_fix_exceptions()</code> ; cache frequent queries.
R-002	STT accuracy for accented Turkish speech	High	High	Expand Whisper test corpus; add post-processing normalisation.
R-003	TTS internet dependency (Edge TTS)	Medium	Low	Switched to local Piper TTS; offline fallback already implemented.
R-004	Context window overflow in long sessions	Medium	Medium	<code>_shrink_chat_history()</code> tested in ST-005; monitor token usage logs.
R-005	PCA dimensionality reduction semantic loss	Medium	High	Document known accuracy trade-off; evaluate <code>recall@10</code> on test queries.
R-006	sqlite_vec extension compatibility	Low	Low	Pin extension version; integration test on all target platforms.
R-007	Camera/frame capture not integrated	Medium	High	Hardcoded path flagged; <code>camera.py</code> integration deferred to Phase 2.